Post-training

Lecture 3: how a base model becomes an assistant

Aarne Talman

3 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	**3 / 5 Nov**	**Post-training**	**Lab 2: tune vs prompt**	**Aarne**
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Lecture overview

1. Base vs instruct *10 min*
2. The post-training pipeline *5 min*
3. Supervised fine-tuning *15 min*
4. Preference optimisation *15 min*
5. RL with verifiable rewards: GRPO *15 min*
6. Parameter-efficient training *10 min*
7. Prompt, retrieve, or fine-tune? *10 min*
8. Lab 2 briefing *10 min*

Take away

By the end of today you should be able to explain how supervised fine-tuning, preference optimisation and verifiable rewards change a base model, and choose a practical method for a small project.

Base vs instruct

The same prompt, two different replies

Base model

```
Give me two numbers that sum
to 10 and multiply to 21.

Give me two numbers that sum
to 12 and multiply to 35.
Give me three numbers that
sum to 9 and multiply to 24.
Solutions are on page 212.
```

Instruct model

```
Give me two numbers that sum
to 10 and multiply to 21.

3 and 7.

(3 + 7 = 10, and 3 x 7 = 21.)
```

Both models share the architecture, the tokeniser and the pretraining data. They differ in **post-training** and in the chat interface built around it.

Why a base model may continue the document

A base model was trained to continue documents, not specifically to answer a user.

- It has no notion of a *turn*, so it does not know whose text comes next
- It has no notion of *being asked*, so a question is just text that questions tend to follow
- It has no reason to *stop*, because documents do not stop after one paragraph

In pretraining data, a line like this one typically appears on a worksheet, so the model continues the worksheet.

What post-training changes

Much of the model's general knowledge comes from pretraining.

Post-training mainly changes which continuation the model considers likely, once it is given a structure that says *a user asked, now the assistant answers, and then it stops*.

This structure is a **string format**, learned with the same next-token objective as everything else.

A CHAT IS A STRING

`apply_chat_template` takes a list of role/content dictionaries and returns one flat string. This string is all the model sees.

```
msgs = [{"role": "system",  "content": "You are a helpful assistant."},
        {"role": "user",    "content": "Give me two numbers that sum to 10 and multiply to 21."}]
tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=True)
```

This returns the following string, including the newlines:

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
Give me two numbers that sum to 10 and multiply to 21.<|im_end|>
<|im_start|>assistant
<think>

</think>
```

The template is a Jinja program shipped with the weights

```
{%- for message in messages %}
    {{- '<|im_start|>' + message.role + '\n' + message.content + '<|im_end|>' + '\n' }}
{%- endfor %}
{%- if add_generation_prompt %}
    {{- '<|im_start|>assistant\n' }}
    {%- if not enable_thinking %}
        {{- '<think>\n\n</think>\n\n' }}
    {%- endif %}
{%- endif %}
```

It lives in `tokenizer_config.json`, under `chat_template`. It is **part of the model**, although it is not stored in the `.safetensors` file, and it **differs between model families**.

Read it before fine-tuning: it shows where the assistant region begins, and therefore where the loss mask should start.

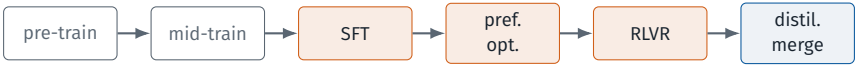
Anatomy of the interface

Piece	What it is
roles	`system`, `user`, `assistant`, and now usually also `tool`. A convention learned during SFT
special tokens	real vocabulary entries (<|im_start|>, <|im_end|>), usually reserved during pretraining and given meaning afterwards
generation prompt	the trailing *open* assistant turn. Without it, nothing tells the model that it should respond
stop condition	the end-of-turn token, which the model is trained to emit and the serving stack is configured to stop on
system prompt	an ordinary turn placed first in the string. It works only because the model was trained to follow instructions in that position

A base model has no `chat_template` field at all. If you apply one anyway, the model receives tokens it has not seen used in this way, and the baseline becomes unreliable.

The post-training pipeline

The 2026 pipeline and what each stage adds



SFT	supervised fine-tuning: teaches the format and *one* good answer per prompt. It cannot express that one answer is better than another, and it needs written responses
pref. opt.	learns which answer is better from pairwise human judgements. Limited by the cost of preference labels
RLVR	reinforcement learning with verifiable rewards: replaces human labels with a program wherever the answer can be checked, so it scales with compute rather than with annotation
distil.	recovers capabilities lost in the RL stages and combines several specialised models into one checkpoint

Later stages can undo earlier ones

According to its technical report, GLM-5 ends with a distillation step *to recover capabilities that may have weakened during the sequential RL stages*.

In other words, **catastrophic forgetting also occurs within the post-training pipeline**. The RL stages for reasoning, agents and general behaviour may weaken abilities learned earlier, and the final step distils them back from earlier checkpoints.

The same effect can appear in a single LoRA run.

Take away

Each post-training stage involves a trade-off. Besides asking whether the target metric improved, we should ask what became worse elsewhere.

In Lab 2 this is measured with a fifty-question general-knowledge check before and after fine-tuning.

Source: Raschka, *Controlling Reasoning Effort in LLMs*, July 2026, quoting the GLM-5 technical report.

Supervised fine-tuning

SFT IS THE PRETRAINING LOSS WITH A MASK

$$\mathcal{L}_{\mathrm{SFT}} \;=\; -\sum_{t=1}^{T} m_t \,\log p_\theta(x_t \mid x_{<t}) \qquad \text{with } m_t \in \{0,1\}$$

This is the next-token loss from week 1 with a weight m_t for each token. When $m_t = 0$, the token is part of the context but does not contribute to the loss.

Compared with pretraining, three things change:

1. the **data** is conversations rendered through the model's own chat template, not web documents
2. the **mask** m_t zeroes the loss on everything that is not assistant content
3. the **budget** is much smaller: thousands of examples on one GPU instead of trillions of tokens on a cluster

The optimiser, the objective and the architecture stay the same. This makes SFT cheap, but it also limits what SFT can change.

The loss mask, drawn



Abbreviations: `im_s` is <|im_start|>, `im_e` is <|im_end|>, `asst` is `assistant`.

The model attends to every token in the sequence, but only the blue cells contribute to the gradient. The final end-of-turn token is **included** in the loss, because the model has to learn when to stop.

What goes wrong if you do not mask

If the loss covers the whole sequence, the model also learns to write the user's turns.

- after its answer, it may go on to generate the next *user* turn
- part of the gradient goes to reproducing prompts you already have
- with long prompts and short answers (classification, NER, extractive QA), most of the loss comes from the inputs
- the training loss looks low, because the prompts are easy to predict

In practice, on Thursday

TRL's `SFTTrainer` applies the mask when you set `assistant_only_loss=True`.

The flag requires the chat template to mark the assistant region. If it does not, the whole sequence can end up in the loss **without a warning**.

To check, print the labels of one batch and confirm that the masked positions cover the prompt and nothing else.

SFT DATA: QUALITY OVER QUANTITY

A common view is that SFT mostly **selects** behaviour the model already has. For that, clean and consistent demonstrations matter more than their number.

- LIMA fine-tuned a 65B model on **1,000** curated examples and remained competitive with far larger instruction sets [Zhou et al. 2023]
- inconsistent examples teach the model that the format is optional
- a large scraped set reflects the average quality of its annotators, including their mistakes

What quality means in practice

consistent format: the same answer shape every time

correct: checked, with a script where possible

on-distribution: the prompts resemble those you expect in use

diverse: covering the input space, not paraphrases of the same input

deduplicated: within the set and against the test set

For Lab 2, **1–5k examples** is a reasonable size. Time spent cleaning the data usually helps more than time spent collecting more of it.

Catastrophic forgetting

What it is. Training on new behaviour changes weights that also encode earlier behaviour, which can then degrade. The effect is well documented in continued pretraining, and SFT works through the same mechanism on a smaller scale.

Why it is hard to avoid. The model has no unused capacity set aside for the new task. Gradients from a narrow distribution change representations that the rest of the model also relies on.

What makes it worse: high learning rate, many epochs, narrow data, full fine-tuning rather than LoRA, and a small model.

What it looks like

The task metric improves by 12 points, but:

The model now answers every question in your task's format, including "what is the capital of Finland".

It no longer handles a second language it used to support.

It no longer refuses anything.

The task metric shows none of this.

Mitigations, roughly in order of cost: fewer epochs • lower learning rate • LoRA with a small rank • mixing general data back in (rehearsal) • distilling from your own pre-SFT checkpoint.

Preference optimisation

Why SFT is not enough

SFT maximises the likelihood of **one** demonstrated answer. It has no way to express that one answer is **better than** another.

That matters whenever the target is a *judgement* rather than a fact:

- helpfulness, tone, length, hedging
- refusing the right things and only those
- "answer the question that was asked"
- formatting preferences that are hard to state as rules

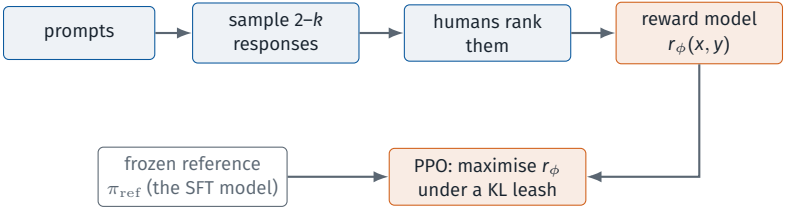
Writing a gold answer for these is hard, but *choosing* the better of two candidates is much easier. The methods in this section build on that difference.

SFT has no negative examples

SFT is trained only on good answers, so nothing in its loss pushes probability *away* from the errors the model actually makes.

The methods in this section add that negative signal in different ways.

RLHF IN ONE DIAGRAM



Some RL terminology. The **policy** is the model being trained. The **reference** is a frozen copy of it from before this stage. The **KL leash** is a penalty for drifting away from that copy. The **value model** (or critic) is a second network that estimates how good an answer will be.

PPO-based RLHF therefore keeps four large models in memory at once, which motivated the search for simpler methods.

Source: Ouyang et al. 2022, *Training language models to follow instructions with human feedback* (InstructGPT).

The reward model is logistic regression on pairs

Collect triples: a prompt x, the answer a human *preferred* (y_w) and the one they *rejected* (y_l). Train a scorer r_ϕ so that

$$P(y_w \text{ wins}) \;=\; \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

The larger the score difference between two answers, the more confident the prediction of which one a human chose. The model is fitted like a logistic regression, by maximising the likelihood of the observed choices. This is the **Bradley–Terry** model, which returns in week 4 in arena leaderboards.

r_ϕ is usually the SFT model with its next-token head swapped for a head that outputs a single number. Only score *differences* are determined, because the scale has no fixed zero.

The RL stage then maximises the average reward from r_ϕ, minus a penalty β times the divergence from the frozen starting model.

PPO IN BRIEF, SINCE GRPO BUILDS ON IT

$$\mathcal{J}_{\text{PPO}} = \mathbb{E}\Big[\min\Big(\rho\hat{A},\ \text{clip}(\rho)\,\hat{A}\Big)\Big], \qquad \rho = \frac{\pi_{\text{new}}(\text{this token})}{\pi_{\text{old}}(\text{this token})}$$

PPO makes the tokens of better-than-average answers more likely and the rest less likely, while limiting how far one batch can move the model.

Advantage $\hat{A}$: how much better than expected this answer was. The expected value has to be estimated.

Clipping: the ratio is kept within about 20% of 1, so that a single bad batch cannot change the model too much.

The critic: a second network, as large as the policy, that estimates the expected value.

Source: Schulman et al. 2017, *Proximal Policy Optimization Algorithms*.

Why this matters for GRPO

The critic is a model of the same size as the policy, with its own loss and its own failure modes.

GRPO estimates the same baseline without a critic.

Reward source and optimisation are separate choices

	Reward source	Reward model?	Critic?
RLHF + PPO	learned human-preference score	yes	yes
GRPO	learned score	yes	**no**
RLVR + GRPO	programmatic verifier	**no**	**no**

- **GRPO removes the critic**: it compares several responses to the same prompt instead of predicting a baseline with another network.
- **RLVR changes the reward source**: a program checks the answer. GRPO can also be used with a learned reward.
- **DPO takes a different route**: it trains on stored preference pairs, without generating responses during the update.

DPO LEARNS DIRECTLY FROM PREFERRED AND REJECTED ANSWERS

A DPO example contains a prompt, a **preferred answer** and a **rejected answer**. Training increases the preferred answer's likelihood relative to the rejected one.

Why it is useful

- people often find comparison easier than writing an ideal answer;
- training resembles ordinary supervised learning;
- there is no separate reward model or rollout loop.

What to watch

- the pair only tells us which answer is better;
- the data may reflect annotator or judge bias;
- a frozen reference model keeps the update close to its starting behaviour.

Take away

Use DPO when you have reliable preference pairs and the target quality is difficult to express as an automatic score.

Source: Rafailov et al. 2023, *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.*

Where preference optimisation is still used

Preference-based rewards are still used, mainly for tasks whose answers **cannot be verified** automatically.

DeepSeek V3.2 uses a rule-based outcome reward, a length penalty and a language-consistency reward *for reasoning and agent tasks*, and a **generative reward model with prompt-specific rubrics** *for general tasks*.

Current pipelines are therefore **hybrid**: verifiers where the answer can be checked, and LLM judges with rubrics elsewhere. **Kimi K3** similarly uses learned reward models and pairwise comparisons for non-verifiable and agentic tasks.

Take away

A useful diagnostic for your own project:

if a model does well on multiple-choice and verifier benchmarks but poorly with an LLM judge or in an arena, it probably knows the answers but does not present them well. This is where preference optimisation is most likely to help.

Sources: Raschka, *From DeepSeek V3 to V3.2*, December 2025; *Understanding the 4 Main Approaches to LLM Evaluation*. Model facts as of October 2026.

RL WITH VERIFIABLE REWARDS: GRPO

Verifiable rewards and reasoning models

The main idea

If the answer can be **checked** automatically, there is no need to model human preferences, and the human labelling that limited post-training is no longer needed for that task.

SFT needs written responses and RLHF needs preference labels. Both are produced by people, at a cost per item.

A verifier removes most per-item human labelling, but generation and checking still cost compute. Post-training therefore shifts much of the budget from annotation to computation.

Names and dates

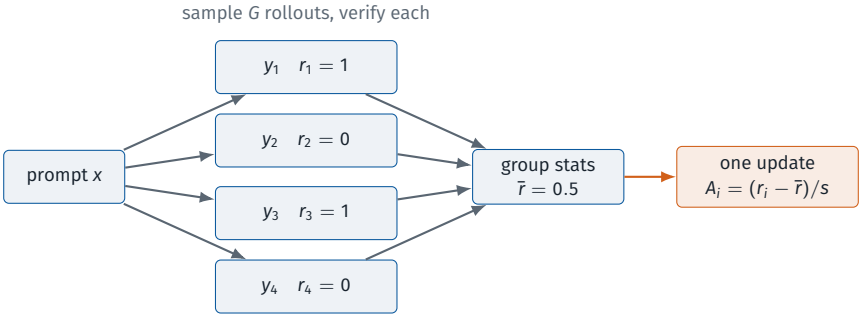
The term **RLVR** was introduced in **Tülu 3**, about two months before DeepSeek-R1.

DeepSeek-R1 and **Kimi K1.5** were posted to arXiv on the same day, **22 January 2025**.

R1 received more attention, mainly because R1-Zero showed reasoning behaviour emerging from RL *without* SFT.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026; *The State of LLMs 2025.*

GRPO: ONE PROMPT, A GROUP OF ROLLOUTS, ONE UPDATE



1. Sample G complete responses for the *same* prompt (a *rollout* is one complete generated response). **2.** Verify each: 1 if correct, 0 if not. **3.** Score each response relative to the rest of *its own group*. **4.** Clipped policy-gradient update.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026, videos 6–7.

GRPO USES THE GROUP AS ITS BASELINE

$$A_i = \frac{r_i - \text{mean reward of the group}}{\text{standard deviation of the group}}$$

The **advantage** measures how much better an answer is than the other answers to the same prompt.

- PPO trains a critic to predict this baseline. GRPO computes it from the group, so no critic is needed.
- Example: rewards $1, 0, 1, 0$ give advantages $+0.87, -0.87, +0.87, -0.87$.
- If every answer receives the same reward, every advantage is zero and the group gives no learning signal.

DeepSeek-R1-Zero: no SFT at all

R1-Zero applied RLVR **directly to the pretrained base model**. No supervised fine-tuning, no demonstrations, no preference data.

The reward was the sum of two terms: **answer correctness** and **output format**. A language-consistency term was added later, in the training of R1, because R1-Zero mixed languages within answers.

R1, the released model, was trained in several stages, starting with a small SFT step on DeepSeek-V3. R1-Zero performed worse, but it was the more important result.

What each reward does

accuracy: is the final answer correct? Scored 1 or 0 by a rule-based checker, or by test cases for code.

format: is the reasoning inside the thinking tags, with the answer where the parser expects it? The tags exist so that the answer can be extracted.

language consistency (R1): penalises mixing languages, which R1-Zero did because nothing in its reward discouraged it.

Sources: Raschka, *Build Your Own Reasoning Model*, October 2026; *From DeepSeek V3 to V3.2*, December 2025. DeepSeek-R1 paper, January 2025.

The reasoning trace is not directly supervised

In R1-Zero, the reward depends on the **final answer** and the **format**. The intermediate reasoning is not scored, not supervised, and not rewarded directly. DeepSeek also tried process-based rewards and reported that they did not help.

Backtracking, self-checking, trying a second approach and abandoning a failed one nevertheless **emerge** from outcome rewards alone.

A likely explanation. Traces that contain self-correction reach the correct answer more often. The update reinforces the *whole sequence*, including the checking steps.

A limitation. The objective does not require the trace to be *faithful*. It only rewards traces that end in correct answers, and these are not the same thing.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026, citing the DeepSeek-R1 paper.

How self-correction can emerge from outcome rewards

```
Q: Give me two numbers that sum to 10 and multiply to 21.

<think>
Let me try 5 and 5. The sum is 10. The product is 25.
Wait - that is not 21. Let me re-check what I need.
I need a + b = 10 and a * b = 21.
The factor pairs of 21 are (1, 21) and (3, 7).
1 + 21 = 22, no. 3 + 7 = 10, yes.
Check the product: 3 * 7 = 21. Both conditions hold.
</think>

3 and 7.
```

In this illustrative trace, the model starts with a wrong guess, **notices** the error and corrects it. The behaviour was not demonstrated in training data or rewarded directly. It was reinforced because traces that contain it end in correct answers more often.

Verifiers in practice

Domain	Checker	Where it breaks
mathematics	SymPy or WolframAlpha for symbolic equivalence	extraction and normalisation: is 0.5 the same answer as $\frac{1}{2}$?
code	a compiler, unit tests, or a platform like LeetCode	tests that pass for the wrong reason; timeouts; sandbox escapes
computer use (2026)	success or failure signals from the environment	building the environments
structured output	a JSON-schema or grammar check	valid syntax, wrong content

Verification usually has three steps: **extract the final answer** → **normalise it** → **check equivalence**. Most of the engineering effort goes into the first two. The same verifier can both evaluate the model and provide the reward during training.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026; *GPT-6 Astra, Looped Transformers, and Hidden Reasoning*.

Why GRPO is optional in Lab 2

Cost. Each step generates G complete responses before any gradient is computed. Generation dominates the run time, and a GRPO step can be one to two orders of magnitude more expensive than an SFT step.

Stability. Runs usually need several known corrections that are not enabled by default in every library. A run can fail while the loss curve still looks normal.

Prerequisite. A reliable verifier, which in practice means a tested answer extractor.

Take away

Optional extension B, mainly for GSM8K-style tasks: GRPO with an exact-match reward, about 100 steps, as a `gpumedium` batch job.

One hundred steps will not produce a reasoning model, but the reward curve should start to rise.

Complete the main exercise first.

Parameter-efficient training

LoRA

LoRA freezes the original weight matrix and learns a low-rank update built from two small matrices:

$$W' = W + \frac{\alpha}{r} BA$$

For example, a 2560×2560 weight matrix has 6.5 million parameters. With rank 16, LoRA instead trains a 2560×16 and a 16×2560 matrix, about 82 thousand parameters.

A is initialised randomly and B with **zeros**, so the update is zero at the start and training begins from the original model.

Trainable parameters for a 4B model at rank 16, on the attention and MLP matrices: roughly **33M**, or about **0.8%**.

Main settings

rank: the capacity of the update. Values from 8 to 64 are common.

alpha: a scaling factor, often set to twice the rank.

target modules: the matrices that receive an adapter. Attention-only is cheaper; Lab 2 uses all linear layers.

Source: Hu et al. 2021, *LoRA: Low-Rank Adaptation of Large Language Models.*

Memory requirements for a 4B model

Full fine-tuning, mixed precision	
weights, bf16	2 bytes/param
gradients, bf16	2 bytes/param
Adam momentum, fp32	4 bytes/param
Adam variance, fp32	4 bytes/param
fp32 master copy	4 bytes/param
total	**16 bytes/param**

For 4 billion parameters this is **64 GB**, *before* activations.

LoRA, bf16	
frozen base weights, bf16	8 GB
adapter weights (33M)	0.07 GB
adapter gradients	0.07 GB
Adam state for 33M	0.26 GB
total state	**≈ 8.4 GB**

Activations add a few GB at sequence length 1024 with gradient checkpointing, so the total stays well under **20 GB**.

A GH200 has **96 GiB**, so full fine-tuning of a 4B model would just fit. LoRA needs about a fifth of that memory, trains faster and typically causes less forgetting.

Merging, and serving adapters

Merge. Computing $W \leftarrow W + \frac{\alpha}{r} BA$ gives an ordinary checkpoint with **no inference overhead**.

- merge for a single deployment and for clean benchmarking
- merging loses nothing except the option to remove the adapter later
- merge in the *same* precision you trained in; otherwise rounding introduces small differences

Keep it separate. An adapter takes tens of megabytes. Serving frameworks can switch adapters per request on top of one shared base model.

- one base model and many adapters on one GPU
- a small amount of extra compute per token
- this is how many fine-tuning services work

In week 3 you can serve your own adapter in this way with vLLM.

For Lab 2: keep the adapter rather than a merged checkpoint. It is small and easy to reload if you use it in later labs.

Prompt, retrieve, or fine-tune?

Prompting, retrieval or fine-tuning

You want to change	Use	Because
behaviour, format, style, tone, output schema, verbosity	**fine-tuning**	training changes the model's output distribution
facts that are private, changing, or need a citation	**retrieval**	model weights are a poor database: hard to update, no sources, no access control
nothing yet; the task is still being defined	**prompting**	no training cost, fast iteration, and you need it as a baseline anyway

The options are often combined. Many production systems use **a fine-tuned model that answers over retrieved context**, which brings together weeks 2 and 5 of this course.

What each one cannot fix

Prompting cannot

add a capability the model does not have.

It also struggles to make a small model reliable on a long multi-step format: the model may follow it for a few turns and then drift.

Retrieval cannot

change behaviour. If the model does not follow your output schema, more context will not help.

Nor does it help if the model ignores the retrieved context.

Fine-tuning is a poor fit for

facts that change or need sources. Fine-tuning can teach facts, but training on new factual knowledge can also **increase** hallucination [Gekhman et al. 2024].

The model may learn to answer confidently in the trained format without knowing the answer.

Use retrieval when facts must be current, attributable or easy to update.

A common and costly mistake is to fine-tune on an internal FAQ. The model learns the *form* of a confident internal answer and then produces policy numbers that look right but are invented.

The order of operations

1. **Write a zero-shot prompt.** Score it on development data. This is your initial baseline.
2. **Improve the prompt.** Few-shot examples, an explicit output format, a system prompt. Score again. It is common to stop here before the prompt is as good as it could be.
3. **Analyse the remaining errors.** Missing *knowledge* $\rightarrow$ retrieval. Wrong *behaviour* or *format* $\rightarrow$ fine-tuning. Both $\rightarrow$ both.
4. **Lock the protocol, then use the held-out test set once.** Compare the selected prompt with the fine-tuned model, including behaviour you did not intend to change.

Take away

Steps 1 and 2 matter. Improving on a weak prompt says little; improving on a strong prompt is a result worth reporting.

For your task, does fine-tuning beat your best prompt?

The question has several parts:

- **your task**: the task from Lab 1 or another suitable task, with its own held-out set
- **your models**: a pretrained base and instruct pair from Hugging Face (Qwen3.5-0.8B by default), not the Week 1 tiny GPT
- **your best prompt**: the best of zero-shot and few-shot prompts on *both* the base and the instruct model, not the first prompt you tried
- **beat**: measured on held-out data that neither model was trained on, with identical decoding settings

A further question is often left out: **what did it cost?** This is why the results table also includes a forgetting check.

Later weeks: **week 4** covers how to evaluate these numbers, **week 5** covers retrieval, and **week 6** builds a tool-using agent.

Lab 2 briefing

Keep a results table with **four rows**:

Setting	Held-out score	Notes
zero-shot, instruct		lower reference point
best prompt		which model, which prompt, how many shots
LoRA fine-tuned		rank, epochs, learning rate
forgetting check		50 general-knowledge questions, before *and* after

Add the training loss curve and a paragraph on whether fine-tuning was *worth it*, in GPU time and in points.

A negative result is also a result. “Fine-tuning did not beat a five-shot prompt, and this is our explanation” is a valid finding if the numbers are sound.

`Week-2/Lab-2/README.md` contains the runnable baseline, LoRA, evaluation and plotting workflow.

The session, and what to do before it

Thursday, 16:15–17:45

0–10	set-up: pull the files, smoke test, install packages
10–25	run the bundled example on a GPU
25–60	your task: data, config, try prompts
60–90	submit the full experiment, read the outputs

The full experiment is one batch job and can take an hour or more, so it may finish after the session.

Before Thursday

Prepare what needs no GPU:

- choose an existing labelled dataset for a classification task, for example SST-2
- make the splits with `make_splits.py` and check them with `check_data.py`
- write the instruction for your task

The steps are in `Week-2/Lab-2/README.md`.

Baselines, and three ways to ruin one

The protocol

- zero-shot *and* few-shot
- base *and* instruct
- fixed decoding settings. Greedy removes sampling variation, although exact GPU repeatability is not guaranteed
- the held-out set is **not** used during development

Choose the best of the four on the development set, report which one it was, and then score only that choice on the held-out test set.

What goes wrong

1. Answer extraction. The model answered correctly, but the regular expression missed it. Read ten raw generations (`show_examples.py`) before trusting a score.

2. Chat template on a base model. Base models have none. Use plain completion prompts with a consistent separator.

3. Contamination. Public benchmarks may be in the pretraining data. Check a suspiciously high zero-shot score before trusting it.

All three affect the *baseline*, which every other result is compared against, so an error here carries over to the later labs.

Measuring, optional extensions, and files to keep

The forgetting check Fifty general-knowledge multiple-choice questions, unrelated to your task. Score the model you started from, score yours, and report the delta, using the same prompt and the same decoding both times.

Optional extensions (Lab 2 README)

A: DPO on a preference set. Start with beta 0.1. The reference model is the same model with the adapter disabled.

B: GRPO with an exact-match reward, about 100 steps, as a `gpumedium` batch job. Best suited to GSM8K-style tasks.

Keep for the final submission

the results table • the loss curve • the **adapter**, not a merged checkpoint • your config, scripts and Slurm script • your prompt file

Roihu reminders

`gputest` before `gpumedium`.

Close idle GPU Jupyter sessions: a forgotten session can hold a GH200 for up to twelve hours.

Build environments on the ARM login nodes.

Three things to remember

1. **A chat is a string, and the chat template defines the interface.** Roles, special tokens and the generation prompt form a format that the model was trained on.
2. **If the answer can be verified, a learned reward model is not needed.** This moved much of the cost of post-training from annotation to compute, and it led to current reasoning models.
3. **Measure what fine-tuning broke, not only what it improved.** GLM-5 added a distillation stage to recover capabilities weakened by RL. The forgetting check in Lab 2 applies the same idea on a small scale.

In all three cases, post-training mainly shapes behaviour, using far less data than pretraining. It can add task-specific information, but it is not a reliable way to store facts that change.

Thursday, and the next two weeks

Thursday 5 November — Lab 2
Does fine-tuning beat your best prompt? Bring formatted data and a working answer extractor.

Week 3 — Aarne Talman, inference
Decoding, KV-cache memory, serving, quantisation and the cost of running your model.

Week 4 — Jussi Karlgren, evaluation: metrics and their limits, contamination, LLM-as-a-judge and evaluation across languages.

Why inference comes next

This week we change a model. Next week we serve one and measure what it costs to run. Reusing your Lab 2 adapter is possible but not required.

You will compare the original and FP8 versions with the same held-out metric as in Lab 2, so speed and quality can be reported in one table.

Reading

The pipeline

- Ouyang et al. 2022, InstructGPT
- Lambert et al. 2024, *Tülu 3*
- Zhou et al. 2023, *LIMA*

Preference optimisation

- Schulman et al. 2017, PPO
- Rafailov et al. 2023, *DPO*
- Ethayarajh et al. 2024, *KTO*
- Hong et al. 2024, *ORPO*
- Meng et al. 2024, *SimPO*

Parameter-efficient training

- Hu et al. 2021, *LoRA*
- Gekhman et al. 2024

RLVR and reasoning

- DeepSeek-AI 2025, *DeepSeek-R1*
- Yu et al. 2025, *DAPO*
- Liu et al. 2025, *Dr. GRPO*
- DeepSeekMath-V2, November 2025

Current practice, Raschka

- *Build Your Own Reasoning Model* (Oct 2026)
- *Controlling Reasoning Effort in LLMs* (Jul 2026)
- *From DeepSeek V3 to V3.2* (Dec 2025)
- *The State of LLMs 2025*

Libraries

- TRL docs: SFT, DPO, KTO, GRPO trainers